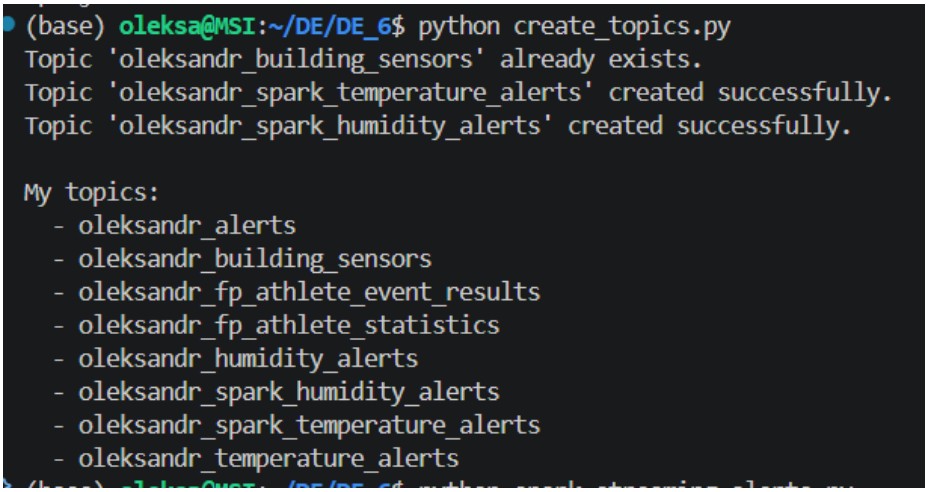
Spark Structured Streaming: алерти IoT-датчиків

Домашнє завдання DE_6 · Oleksandr · 18.10.2026

1. Генерація потоку даних

Використано топік і генератор з DE_5. create_topics.py: вхідний топік oleksandr_building_sensors вже існує, створено два вихідні топіки алертів:

```
- oleksandr_spark_temperature_alerts  - алерти 103/104 (температура)
- oleksandr_spark_humidity_alerts     - алерти 101/102 (вологість)
```

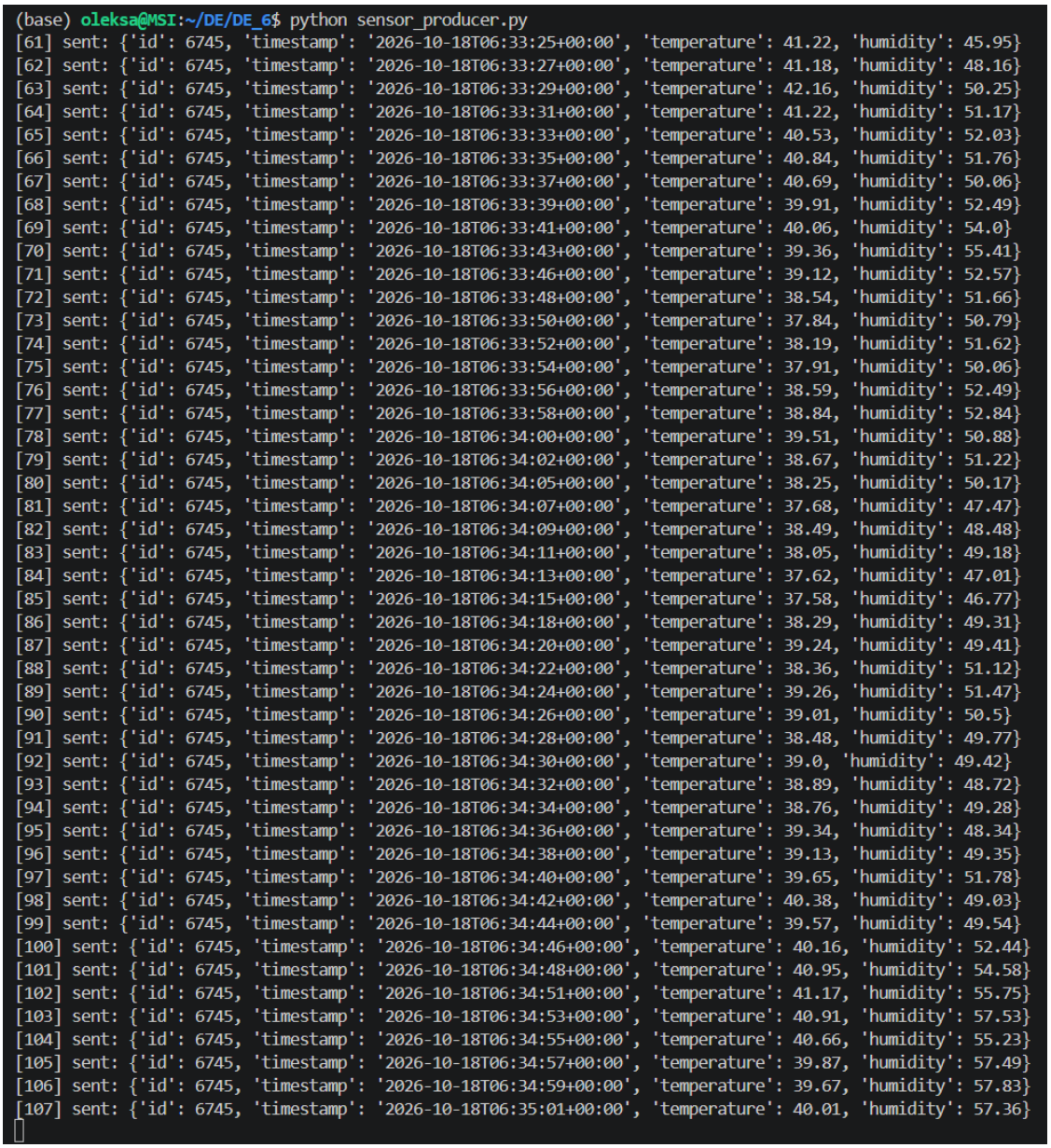


Скрін: topics.png

sensor_producer.py запущено двічі одночасно в окремих терміналах -> два датчики з ID 6745 та 7859. Кожні ~2 с у building_sensors відправляється повідомлення з полями id, timestamp, temperature, humidity:

```
{'id': 7859, 'timestamp': '2026-10-18T06:33:20+00:00', 'temperature': 42.27, 'humidity': 50.88}
```

Значення випадкові в межах 25..45 °C та 15..85 %, але генеруються випадковим блуканням (плавна зміна, як у реального датчика). При незалежних рівномірних значеннях (як у DE_5) середнє за 1 хв від ~60 вимірювань практично завжди ≈35 °C / ≈50 % і жодна умова алерту не виконується.



Скрін: sensor_producer_id_6745.png

```
(base) oleksa@MSI:~/DE/DE_6$ python sensor_producer.py
[55] sent: {'id': 7859, 'timestamp': '2026-10-18T06:32:57+00:00', 'temperature': 40.7, 'humidity': 51.44}
[56] sent: {'id': 7859, 'timestamp': '2026-10-18T06:32:59+00:00', 'temperature': 41.5, 'humidity': 52.6}
[57] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:01+00:00', 'temperature': 41.06, 'humidity': 54.96}
[58] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:03+00:00', 'temperature': 41.25, 'humidity': 56.22}
[59] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:05+00:00', 'temperature': 40.91, 'humidity': 54.84}
[60] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:07+00:00', 'temperature': 40.5, 'humidity': 55.61}
[61] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:09+00:00', 'temperature': 40.92, 'humidity': 52.74}
[62] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:12+00:00', 'temperature': 40.08, 'humidity': 50.21}
[63] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:14+00:00', 'temperature': 39.69, 'humidity': 51.64}
[64] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:16+00:00', 'temperature': 40.48, 'humidity': 49.78}
[65] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:18+00:00', 'temperature': 41.37, 'humidity': 50.12}
[66] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:20+00:00', 'temperature': 42.27, 'humidity': 50.88}
[67] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:22+00:00', 'temperature': 42.27, 'humidity': 52.35}
[68] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:25+00:00', 'temperature': 42.03, 'humidity': 55.33}
[69] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:27+00:00', 'temperature': 41.25, 'humidity': 53.06}
[70] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:29+00:00', 'temperature': 40.32, 'humidity': 52.63}
[71] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:31+00:00', 'temperature': 41.23, 'humidity': 49.98}
[72] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:33+00:00', 'temperature': 41.91, 'humidity': 52.12}
[73] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:35+00:00', 'temperature': 42.08, 'humidity': 54.78}
[74] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:37+00:00', 'temperature': 42.77, 'humidity': 54.99}
[75] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:39+00:00', 'temperature': 43.41, 'humidity': 53.67}
[76] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:41+00:00', 'temperature': 43.08, 'humidity': 55.62}
[77] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:43+00:00', 'temperature': 42.93, 'humidity': 53.56}
[78] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:46+00:00', 'temperature': 43.51, 'humidity': 52.77}
[79] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:48+00:00', 'temperature': 43.13, 'humidity': 53.67}
[80] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:50+00:00', 'temperature': 43.82, 'humidity': 51.58}
[81] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:52+00:00', 'temperature': 44.23, 'humidity': 49.23}
[82] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:54+00:00', 'temperature': 44.84, 'humidity': 51.7}
[83] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:56+00:00', 'temperature': 43.93, 'humidity': 49.03}
[84] sent: {'id': 7859, 'timestamp': '2026-10-18T06:33:58+00:00', 'temperature': 44.84, 'humidity': 49.86}
[85] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:00+00:00', 'temperature': 44.79, 'humidity': 51.1}
[86] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:02+00:00', 'temperature': 44.87, 'humidity': 50.43}
[87] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:04+00:00', 'temperature': 44.39, 'humidity': 49.79}
[88] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:06+00:00', 'temperature': 44.86, 'humidity': 49.8}
[89] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:08+00:00', 'temperature': 44.61, 'humidity': 51.03}
[90] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:10+00:00', 'temperature': 43.73, 'humidity': 50.63}
[91] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:12+00:00', 'temperature': 44.31, 'humidity': 48.74}
[92] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:14+00:00', 'temperature': 44.81, 'humidity': 50.14}
[93] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:17+00:00', 'temperature': 44.38, 'humidity': 48.21}
[94] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:19+00:00', 'temperature': 44.54, 'humidity': 45.67}
[95] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:21+00:00', 'temperature': 44.72, 'humidity': 46.47}
[96] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:23+00:00', 'temperature': 44.45, 'humidity': 46.66}
[97] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:25+00:00', 'temperature': 44.78, 'humidity': 48.14}
[98] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:27+00:00', 'temperature': 44.8, 'humidity': 46.27}
[99] sent: {'id': 7859, 'timestamp': '2026-10-18T06:34:30+00:00', 'temperature': 44.44, 'humidity': 44.7}
```

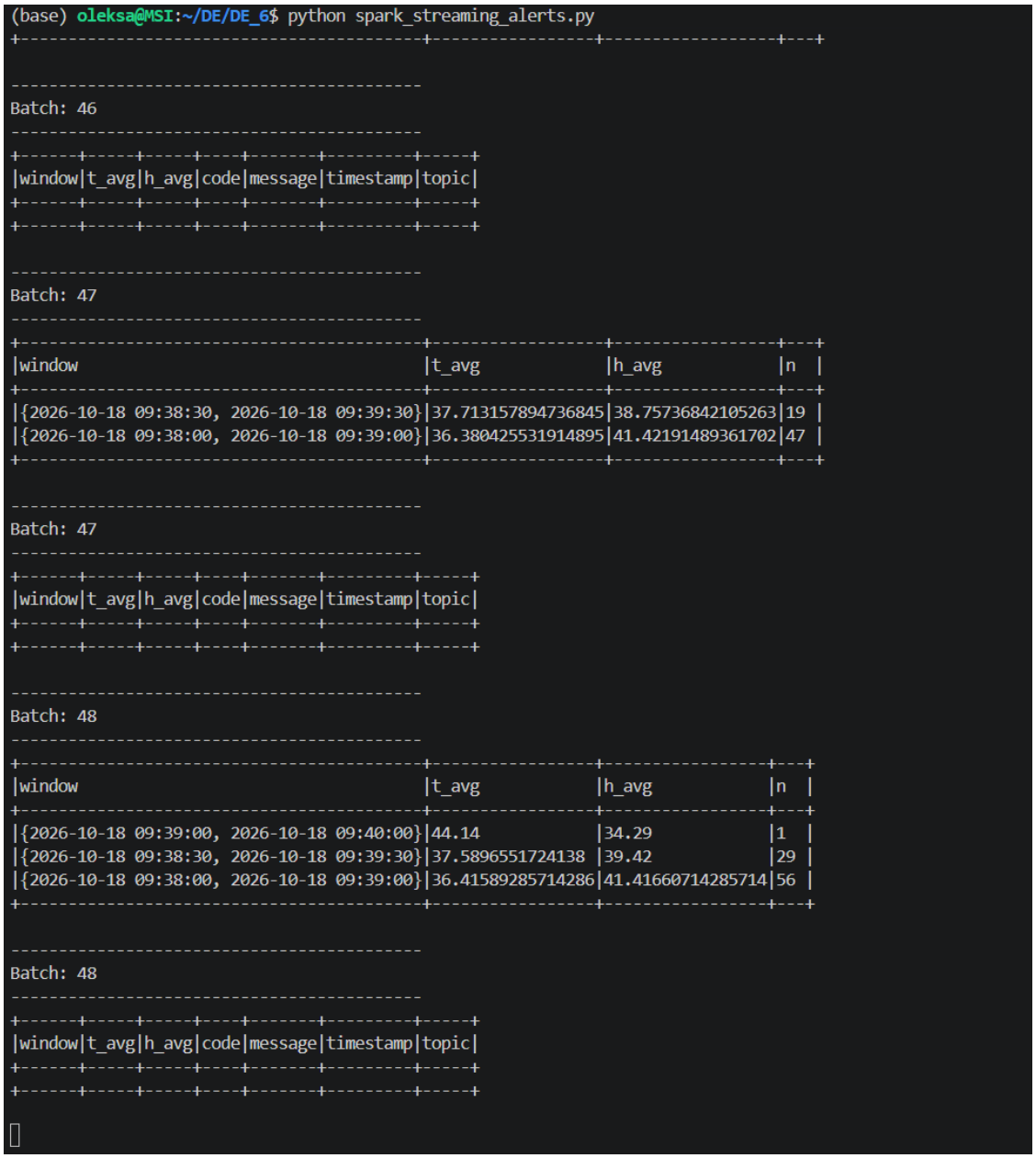
Скрін: sensor_producer_id_7859.png

2. Агрегація даних

spark_streaming_alerts.py читає потік з Kafka (readStream, format kafka), розбирає JSON і рахує середні значення:

```
withWatermark("timestamp", "10 seconds")
.groupBy(window("timestamp", "1 minute", "30 seconds"))
.agg(avg("temperature") as t_avg, avg("humidity") as h_avg, count as n)
```

На скріні — таблиця середніх по вікнах (оновлюється кожні 10 с, режим update): кожна подія потрапляє у два вікна, що перекриваються (09:38:00–09:39:00 та 09:38:30–09:39:30); n — кількість вимірювань у вікні від обох датчиків. Порожні таблиці між ними — батчі запиту алертів: вікна ще не закриті watermark-ом.



Скрін: spark_streaming_alerts.png

3. Параметри алертів

Критерії зчитуються з alerts_conditions.csv (spark.read.csv), тому їх можна змінювати без зміни коду:

```
id,humidity_min,humidity_max,temperature_min,temperature_max,code,message
1,0,40,-999,-999,101,"It's too dry"
2,60,100,-999,-999,102,"It's too wet"
3,-999,-999,-300,30,103,"It's too cold"
4,-999,-999,40,300,104,"It's too hot"
```

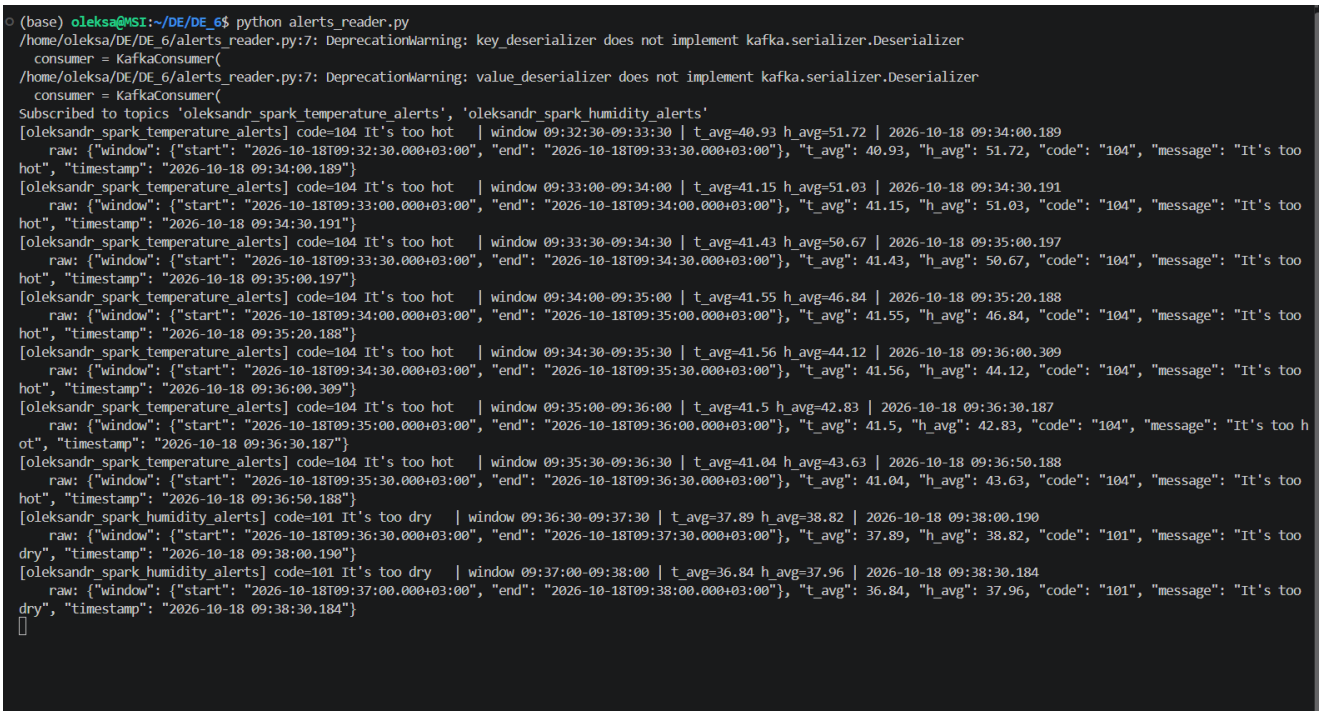
4. Визначення алертів

Середні значення об'єднуються з усіма умовами через cross join, після чого фільтруються: значення має бути в межах [min, max], а межа -999 ігнорується:

```
(min == -999 OR avg >= min) AND (max == -999 OR avg <= max)
  - для h_avg з humidity_min/max та для t_avg з temperature_min/max
```

5. Запис у Kafka

Алерти записуються у Kafka (writeStream, format kafka, режим append — кожне вікно публікується один раз після закриття watermark-ом). Колонка topic маршрутизує повідомлення: коди 101/102 -> humidity-топік, 103/104 -> temperature-топік; key = code, value = JSON. alerts_reader.py читає обидва топіки і показує отримані повідомлення:



Скрін: alerts_reader.png

Приклад повідомлення в Kafka:

```
{"window": {"start": "2026-10-18T09:33:00.000+03:00", "end": "2026-10-18T09:34:00.000+03:00"},
 "t_avg": 41.15, "h_avg": 51.03, "code": "104", "message": "It's too hot",
 "timestamp": "2026-10-18 09:34:30.191"}
```

У період 09:32:30–09:36:30 обидва датчики показували 40–45 °C, тому для кожного вікна t_avg > 40 і спрацював алерт 104 «It's too hot» -> топік oleksandr_spark_temperature_alerts. Далі вологість знизилась: у вікнах 09:36:30–09:37:30 (h_avg = 38.82) та 09:37:00–09:38:00 (h_avg = 37.96) спрацював алерт 101 «It's too dry» -> топік oleksandr_spark_humidity_alerts. Тобто відфільтровані алерти потрапляють у відповідні топіки.

```
{"window": {"start": "2026-10-18T09:37:00.000+03:00", "end": "2026-10-18T09:38:00.000+03:00"},
 "t_avg": 36.84, "h_avg": 37.96, "code": "101", "message": "It's too dry",
 "timestamp": "2026-10-18 09:38:30.184"}
```

Висновок

Реалізовано потоковий конвеєр: датчики (Kafka producers) -> building_sensors -> Spark Structured Streaming (sliding window 1 хв / 30 с, watermark 10 с) -> cross join з умовами з CSV та фільтрація -> алерти у вихідні Kafka-топіки. Критерії алертів винесено в CSV, тож їх можна змінювати без редеплою коду.